EE1003 Introduction to Computer I

Programming Assignment 3:

Big Number Energy Distribution Calculator

Due Date: 2025/12/11 (Thu.) 23:59:59

1. Background

In a distant interstellar civilization, energy technology has evolved to rely on an energy distribution matrix for control and computation. Each energy node can generate, absorb, or transfer energy to its neighbors, and no node exists independently because each node's energy value is influenced by the surrounding nodes.

To simulate this complex interaction, scientists designed an "Energy Distribution Calculator program" to compute the final energy value at each node after considering neighboring influences and weighting effects. However, when the matrix values or weights become extremely large, the accumulated results can easily exceed the range of standard integer data types.

In the C++ language, we know that a long long int variable uses 64 bits (8 bytes) to store data, and the representable range is $[-2^{63}, 2^{63} - 1]$. Even an unsigned long long int can only represent a maximum value of $2^{64} - 1$. So how can we represent numbers greater than $2^{64} - 1$? The answer is Big Numbers.

This programming assignment requires you to implement a "**Big Number Energy Distribution Calculator**" in C++. Each big number can contain up to 100 digits, meaning that the result of a multiplication operation may reach 200 digits, and repeated accumulation can further increase the number of digits depending on the total number of terms. This allows scientists to accurately compute the new energy distribution matrix after mutual interactions between energy nodes and the weighting template, with high precision maintained during addition and multiplication operations.

2. Description

This programming assignment requires you to simulate a Big Number Energy Distribution Matrix Computing System in C++. The program will read an input energy distribution matrix and a corresponding energy weighting template matrix, then perform a series of multiply–accumulate (MAC) operations between the two to generate a new energy distribution matrix. The system must support calculations using big numbers to ensure high precision in energy accumulation and multiplication, even when the energy values or weighting factors become extremely large.

It begins by prompting the user to specify the base-k , where the value of base-k ranges from 2 to 16. After selecting the desired base, the user inputs the size N of the energy distribution matrix, followed by $N \times N$ energy values that represent the energy level at each node. The user then provides the size D of the energy weighting template matrix, along with its $D \times D$ weight values. Once all data are entered, the program performs the computation, applying the MAC operations between the energy matrix and the weighting template, and then displays the resulting energy distribution matrix using the same specified base. At the end of the computation, the program outputs the calculated result and then terminates its execution. The calculator must strictly follow the rules of the four basic arithmetic operations, and no parentheses should appear in any user input. All numerical data must be stored and processed using appropriate data structures such as arrays, vectors, or strings to ensure precision and scalability.

3. Energy Distribution Computation Mode

The energy distribution matrix performs MAC operations with the weighting template to generate the new matrix. The computation involves the following core components:

- Energy Distribution Matrix: The matrix serves as the primary data structure for simulating how energy is distributed and interacts within the field.
- Energy Weighting Template Matrix: This matrix defines the influence of each energy node on its neighbors. Its elements act as weighting factors that adjust the strength of local energy interactions during computation.
- Local Matrix: A small $D \times D$ matrix used to capture local energy

interactions.

- Stride: The step size for sliding the local matrix (fixed to 1).
- Padding: Zeros are added around the matrix to prevent out-of-bound errors and maintain output size.

$$Padding = \frac{Energy\ Weighting\ Matrix\ size - 1}{2}$$

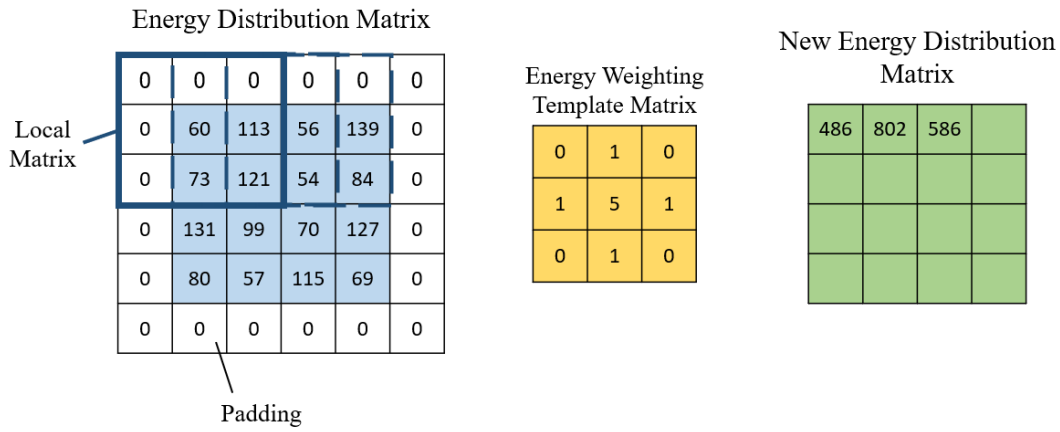


Fig. 1 Energy Distribution Computation

For each position, the local matrix overlaps with a region of the energy distribution matrix. The corresponding elements are multiplied and then summed to obtain one new energy value. The local matrix then slides to the next position, repeating the process until the entire matrix is covered. As a result, the output matrix represents the new energy field after local accumulation and weighting.

4. Input Format

1. The first line: an integer K the value of base-k (ranges from 2 to 16).
2. The next line: an integer N, the size of the energy distribution matrix ($N \times N$).
3. The next N lines: $N \times N$ energy values (integer, space-separated; for bases greater than 10, use uppercase letters A, B, C, … to represent digits above 9).
4. The next line: an integer D, the size of the weighting template matrix ($D \times D$).

5. The next D lines: $D \times D$ weight values (integer, space-separated; for bases greater than 10, use uppercase letters A, B, C, … to represent digits above 9).

5. Output

The program will start by displaying a welcome message, followed by a prompt for the user to specify the base-k. After that, it will prompt the user to enter the size and values of the energy distribution matrix and the weighting template matrix. Once all data are entered, the program will perform the computation and display the exact result after the calculation. Finally, it will output the new $N \times N$ energy distribution matrix(integer, space-separated) that represents the updated energy distribution matrix. The programmer needs to provide fool-proof validation for all input steps. When the program finishes computation, it will display the result and then terminate execution.

6. Limited

In this assignment, several restrictions are applied to ensure the stability and reliability of the Big Number Energy Distribution Calculator.

1. Each big number input can have up to 100 digits, allowing the program to perform large MAC operations. Since **multiplying two 100-digit numbers can produce results of up to 200 digits**, and **repeated accumulation may further increase the length by approximately** $\boldsymbol{log_{10}(n)}$, the system is designed to handle such large intermediate and final MAC values within reasonable memory and computational limits.
2. When displaying the resulting energy distribution matrix, **all elements must be left-aligned**, and each column should have a fixed width equal to **the maximum number length among all results plus two spaces** to maintain neat and readable output formatting.
3. If the user enters an invalid or wrong input, the program will show "**`Invalid input! Please try again.`**". After displaying the message, the user will be prompted to re-enter the input.

7. Sample

Note: Italic shows the input from the keyboard.

```
---- Big Number Energy Distribution Calculator ----
--------------------------------------------------
Enter the base (2 <= k <= 16): base-k = 10
Enter N (size of energy matrix, N x N): 4
Now enter the 4x4 energy matrix (base-10):
Row 1 (enter 4 values separated by spaces, then press
Enter):
10000000000 10000000000 10000000000 10000000000
Row 2 (enter 4 values separated by spaces, then press
Enter):
10000000000 10000000000 10000000000 10000000000
Row 3 (enter 4 values separated by spaces, then press
Enter):
10000000000 10000000000 10000000000 10000000000
Row 4 (enter 4 values separated by spaces, then press
Enter):
10000000000 10000000000 10000000000 10000000000
  //There is a blank line here
Enter D (size of weight matrix, D x D): 3
Now enter the 3x3 weight matrix (base-10):
Row 1 (enter 3 values separated by spaces, then press
Enter):
1 1 0
Row 2 (enter 3 values separated by spaces, then press
Enter):
2 3 3
Row 3 (enter 3 values separated by spaces, then press
Enter):
4 4 4
```

```
  //There is a blank line here
--- Computation complete: Energy distribution result
(4x4, base-10) ---
140000000000  200000000000  200000000000  130000000000
150000000000  220000000000  220000000000  150000000000
150000000000  220000000000  220000000000  150000000000
70000000000   100000000000  100000000000  70000000000
  //There is a blank line here
Program finished.
```

8. Bonus

Please implement the following two additional calculator functions:

1. **Extended Energy Distribution Calculator:**

   Following the basic requirements, can your program handle energy values that include negative integers?

2. **Global Energy Averaging:**

   Based on the basic task, after performing the accumulation operation between the energy distribution matrix and the energy weighting template, the program can further carry out a Global Energy Averaging process.

   This step sums up all the energy values in the final energy field matrix and divides the result by the total number of elements to obtain a single average value that represents the overall energy state. The main purpose of this step is to compress the entire energy distribution map into a single representative value, reducing data volume and making it easier to analyze the overall energy trend. It also helps to keep the final output energy value within a reasonable range and avoid deviation caused by excessively large or small energy values.

   The averaging operation is performed by "adding all elements and dividing by the total number of elements." For example, for a 5×5 Energy Distribution Matrix Y, add all 25 values and divide by 25 to get the global average value.

$$Y_{avg} = \frac{i}{H \times W} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} Y(i,j)$$

Let H and W represent the height and width of the output matrix.

If the division cannot be perfectly divided, the result should be rounded to the nearest integer. (The rounding process should be done using integer comparison: check whether **2 × remainder ≥ divisor**.)

9. Bonus Sample

```
---- Big Number Energy Distribution Calculator ----
--------------------------------------------------
Enter the base (2 <= k <= 16): base-k = 10
Enter N (size of energy matrix, N x N): 4
Now enter the 4x4 energy matrix (base-10), rows one by
one:
Row 1 (enter 4 values separated by spaces, then press
Enter):
10000000000 10000000000 10000000000 10000000000
Row 2 (enter 4 values separated by spaces, then press
Enter):
-30000000000 -200000000000 -4500000000 -3000000000000
Row 3 (enter 4 values separated by spaces, then press
Enter):
10000000000 10000000000 10000000000 10000000000
Row 4 (enter 4 values separated by spaces, then press
Enter):
-1234567890123 -123123123123 -1111111111 10000000000
  //There is a blank line here
Enter D (size of weight kernel, D x D): 3
Now enter the 3x3 weight matrix (base-10), rows one by
one:
Row 1 (enter 3 values separated by spaces, then press
```

```
Enter):
1 0 1
Row 2 (enter 3 values separated by spaces, then press
Enter):
0 -1 0
Row 3 (enter 3 values separated by spaces, then press
Enter):
-1 2 -2
   //There is a blank line here

--- Computation complete: Energy distribution result
(4x4, base-10) ---   // The output matrix must be
properly aligned (see Fig.2 for reference). When
calculating the maximum number length, the negative
sign must also be included.

330000000000   -371000000000   6181000000000   -
6005500000000

40000000000    210000000000   14500000000
3020000000000

-2432889534000  946043866099   -3109099099099
6611111111

1244567890123   143123123123   21111111111    0

   //There is a blank line here

Global average:

14904279279

   //There is a blank line here

Program finished.
```

```
--- Computation complete: Energy distribution result (4x4, base-10) ---
330000000000    -371000000000   6181000000000   -6005500000000
40000000000     210000000000    14500000000     3020000000000
-2432889534000  946043866099    -3109099099099  6611111111
1244567890123   143123123123    21111111111     0
```

Fig. 2 Output Sample

10. Submission Requirement

You have to submit a source code file (not the entire project) named as StudID_PA3.cpp (ex:9862534_PA3.cpp) and a report named StudID_Name_PA3_report.pdf (ex: 9862534_陳聿廣_PA3_report.pdf). If you implement a bonus version, please submit the source code files separately, but include both versions in a single report file for submission. Name your source code file of the bonus version as StudID_Name_PA3_bonus.cpp (ex: 9862534_陳聿廣_PA3_bonus.cpp) and upload to ee-class with the original version. Note that the only acceptable report file format is .pdf, no .doc/.docx or other files are acceptable. **BE SURE to follow the naming rule mentioned above. Otherwise, your program will not be graded.**

We don't restrict the report format and length. In your report, you must at least describe:

1. How to compile and execute your program; (You can use screenshot to explain)
2. The completion of the assignment; (If you complete all requirements, just specify all)
3. The hardness of this assignment and how you overcome it;
4. Bonus function(s) you implement if any.
5. Any suggestions about this programming assignment? You can also put anything related to the PA in your report, such as pseudocode, control flow diagram, programming developing thought, etc.

11. Grading

The grading is as follows:

1. Correctness of your code: 50%
2. Readability of your code: 10%
3. The report: 10%
4. Demo session: 30%
5. Bonus (at most): 10%

Please submit your assignment on time. Otherwise, the penalty rule will apply:

1. Within 72 hrs delay: 20% off
2. More than 3 days: 0 point

Be sure to attend a demo session (the time will be announced later). If you have questions, please Email to both teacher (andyygchen@ee.ncu.edu.tw) and TA 楊舒晴 (114521026@cc.ncu.edu.tw).